

# FINAL MASTER PROMPT

## Advanced Document Data Extractor — 24-Hour AI Agent Challenge

Act as a senior AI/ML engineer, Python architect, multimodal document-processing engineer, software architect, and hackathon reviewer.

Build a **complete, runnable, maintainable, and reviewer-friendly Document Data Extractor (Advanced)**.

The project must prioritize:

**correctness → validation → reproducibility → maintainability → free-tier efficiency → simplicity**

Do not over-engineer the system.

The objective is to deliver a functioning end-to-end agent that a reviewer can clone from GitHub, configure with a Groq API key, run with simple commands, upload sample documents, and inspect validated structured JSON.

---

## 1. PROJECT OBJECTIVE

Build an AI agent that takes messy business documents such as:

- invoices
- receipts
- purchase orders

in:

- PDF
- PNG
- JPG
- JPEG

and extracts important structured information into clean JSON.

The agent must:

1. Read the document.
2. Determine an appropriate extraction route.
3. Extract structured fields.
4. Normalize values deterministically.

5. Validate the structure with Pydantic.
6. Perform deterministic business sanity checks.
7. Reconcile multi-page information.
8. Return validated structured JSON.
9. Show warnings/errors clearly.
10. Never silently fabricate missing information.

The one-sentence agent definition is:

The agent takes a messy PDF or image document and produces structured, validated JSON containing the document's important business fields.

---

## 2. CORE REQUIRED OUTPUT

The final extraction must support these fields when available:

```
document_type
vendor_name
date
invoice_id
currency
line_items
subtotal
discount
shipping_amount
tax_amount
total_amount
overall_confidence
warnings
```

Each line item must contain:

```
description
quantity
unit_price
total_price
```

Optional fields may be omitted when unavailable.

Required fields must use a defined null policy when genuinely unreadable.

Never invent a value simply because the schema expects it.

---

### 3. APPROVED TECHNOLOGY STACK

Use exactly the following core stack.

#### Language

Python 3.11+ or another currently compatible supported version.

Verify dependency compatibility before finalizing `requirements.txt`.

#### LLM

Groq API.

Default model:

```
qwen/qwen3.6-27b
```

Do not replace this with Qwen 3.8 or another model.

Keep the model configurable through:

```
GROQ_MODEL=qwen/qwen3.6-27b
```

Do not scatter the model ID through the codebase.

#### SDK

Official Groq Python SDK.

#### Validation

Pydantic v2.

#### PDF Processing

PyMuPDF.

Do not use `pdf2image`.

## Image Processing

Pillow.

## UI

Streamlit.

## Environment Configuration

python-dotenv.

## Testing

pytest.

Do not add unnecessary frameworks.

---

# 4. MAINTAINABLE PROJECT STRUCTURE

Use a package-based architecture rather than putting all Python files in the repository root.

Use this structure:

```
document-data-extractor/  
|  
├─ README.md  
├─ requirements.txt  
├─ .env.example  
├─ .gitignore  
|  
├─ src/  
|   └─ document_extractor/  
|       ├── __init__.py  
|       ├── app.py  
|       ├── config.py  
|       ├── models.py  
|       ├── validators.py  
|       ├── extractor.py  
|       ├── pdf_processor.py  
|       ├── image_processor.py  
|       └── text_router.py
```

```
|
|   |— normalizer.py
|   |— merger.py
|   |— error_handlers.py
|   |— prompts.py
|
|— tests/
|   |— __init__.py
|   |— test_models.py
|   |— test_validators.py
|   |— test_normalizer.py
|   |— test_text_router.py
|   |— test_merger.py
|   |— test_image_processor.py
|
|— samples/
|   |— invoice_layout_a.pdf
|   |— invoice_layout_b.png
|   |— receipt_layout.png
|   |— purchase_order_layout.pdf
|
|— expected_outputs/
|   |— invoice_layout_a.json
|   |— invoice_layout_b.json
|   |— receipt_layout.json
|
|— docs/
|   |— architecture.md
```

Maintain strict separation of responsibilities.

---

## 5. MODULE RESPONSIBILITIES

`app.py`

Streamlit UI only.

Do not put extraction implementation directly into the UI.

`config.py`

Load environment variables and centralize configuration.

## `models.py`

Pydantic models and schema definitions.

## `validators.py`

Deterministic business validation and financial sanity checks.

## `extractor.py`

Groq API integration and extraction orchestration.

The LLM provider must be isolated here as much as practical.

## `pdf_processor.py`

PDF:

- opening
- page counting
- native text extraction
- page rendering

## `image_processor.py`

Image:

- validation
- orientation correction
- resizing
- conversion
- compression/encoding preparation

## `text_router.py`

Deterministic decision:

```
native PDF text → usable?  
YES → text extraction  
NO → vision fallback
```

## normalizer.py

Deterministic normalization of:

- numbers
- currency
- dates
- empty/null-like values

## merger.py

Multi-page:

- document field reconciliation
- line-item aggregation
- duplicate prevention
- final merge

## error\_handlers.py

Centralized user-safe error conversion where useful.

## prompts.py

Keep the compact model instructions separate from application logic.

This makes future prompt modification easy.

---

# 6. HYBRID EXTRACTION ARCHITECTURE

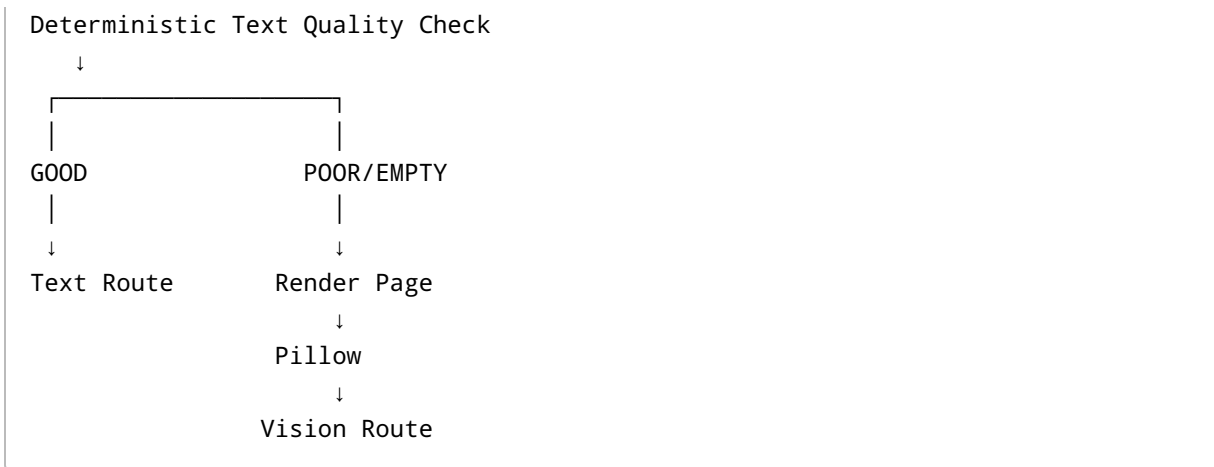
The PDF workflow must use:

## Method 3 — Hybrid text extraction + vision fallback

Do not convert every PDF page to an image automatically.

For each PDF page:

```
PDF Page
↓
PyMuPDF Native Text Extraction
↓
```



This is the primary free-tier optimization.

## 7. TEXT QUALITY ROUTING

Do not assume:

text exists = text is useful.

A page can contain text but still have unusable table/column extraction.

Create a deterministic text quality evaluation using factors such as:

- character count
- word count
- readable-character ratio
- numeric content
- financial keywords
- likely line-item presence
- suspicious fragmentation
- excessive extraction noise

Useful keywords include:

invoice  
receipt  
subtotal  
tax  
total  
amount  
quantity  
qty



```
description
price
unit
PO
purchase order
```

Do not use another LLM to determine route selection.

The router must be deterministic.

Make the threshold configurable.

---

## 8. ROUTING BEHAVIOR

A page should use native text extraction when:

- useful text is available
- important financial data appears extractable
- table information is sufficiently usable

A page should use vision fallback when:

- native text is empty
- native text is extremely poor
- table/column information appears corrupted
- the document is effectively scanned
- extracted text is insufficient for reliable structured extraction

For a mixed PDF, some pages may use text and others vision.

The system must support this.

---

## 9. MULTI-PAGE PDF PROCESSING

Support multi-page PDFs.

Do not process unlimited pages.

Use:

```
MAX_PDF_PAGES
MAX_VISION_PAGES
```

Recommended development defaults:

```
MAX_PDF_PAGES=10
MAX_VISION_PAGES=5
```

Make them configurable.

If total pages exceed `MAX_PDF_PAGES` :

- reject or clearly limit processing according to the chosen implementation
- tell the user exactly what happened

If the number of pages requiring vision exceeds `MAX_VISION_PAGES` :

- do not silently process beyond the budget
- clearly report skipped pages
- mark the result as partial/incomplete where appropriate

Never present a partial extraction as a complete document extraction.

---

## 10. VISION REQUEST POLICY

For the vision route:

One page per LLM request.

Do not send the entire multi-page document as a single visual request.

Do not use multi-image batching for the default implementation.

Reasons:

- predictable token usage
- easier rate-limit management
- easier debugging
- page-level failure isolation
- free-tier safety

One page → one vision request.

---

## 11. TEXT ROUTE EFFICIENCY

When native PDF text is good:

- do not create image tokens unnecessarily
- remove obvious extraction noise locally
- preserve financial values
- preserve dates
- preserve IDs
- preserve line-item text

For text-based pages, combine compact page text into a limited number of model calls only when it remains reliable and token-efficient.

Do not combine pages if this compromises extraction accuracy.

The vision route remains one page per request.

---

## 12. IMAGE PREPROCESSING

For standalone images and PDF pages routed to vision:

1. Validate image.
2. Correct EXIF orientation where appropriate.
3. Convert to RGB.
4. Preserve aspect ratio.
5. Resize if excessively large.
6. Cap longest edge around 1024 pixels.
7. Avoid unnecessary upscaling.
8. Encode efficiently for Groq.
9. Reject/handle corrupted images safely.

The 1024-pixel limit is for:

- payload safety
- memory stability
- predictable processing

Do not falsely claim that resizing directly reduces Groq's billed image-token accounting.

---

## 13. GROQ CONFIGURATION

Use the official Groq Python client.

Load:

```
GROQ_API_KEY
```

from environment variables.

Never hard-code the API key.

Never log the API key.

Use:

```
GROQ_MODEL=qwen/qwen3.6-27b
```

as configurable configuration.

---

## 14. FREE-TIER TOKEN/API OPTIMIZATION

The architecture must be designed around free-tier usage.

Minimize:

- number of LLM requests
- image requests
- unnecessary prompt text
- output tokens
- retries
- repeated processing

Use:

```
reasoning_effort="none"
```

Do not request chain-of-thought.

Do not request explanations.

Do not ask the model to explain arithmetic.

Do not use an LLM for:

- date validation
- financial calculations
- normalization
- schema validation
- text-quality routing
- page hashing
- duplicate detection

Those are deterministic application tasks.

---

## 15. OUTPUT TOKEN CONTROL

Use a conservative but sufficient:

```
max_completion_tokens
```

The setting must be configurable.

Do not set it so low that valid invoices cannot be represented.

Do not set it unnecessarily high.

The JSON output must contain only the fields required by the extraction contract.

No explanatory prose.

No markdown.

No reasoning.

No repeated fields.

---

## 16. REASONING CONFIGURATION

Use:

```
reasoning_effort="none"
```

because this application requires extraction, not a reasoning transcript.

Do not request chain-of-thought.

The model should perform internal processing but return only the requested structured result.

---

## 17. JSON OUTPUT

Use Groq JSON Object Mode:

```
response_format={"type": "json_object"}
```

The system prompt must also explicitly require JSON.

The normal expected output is one JSON object.

Do not rely on JSON Object Mode as the only validation mechanism.

Pipeline:

```
LLM
↓
JSON parsing
↓
normalization
↓
Pydantic
↓
business validation
```

Do not inject the full `model_json_schema()` into every prompt.

Keep the prompt compact.

---

## 18. OPTIONAL FIELD TOKEN OPTIMIZATION

For optional values not found in the document:

- omit them from the model output
- let Pydantic/default logic interpret them as `None`

Do not unnecessarily emit many null fields.

For required fields:

- if readable → return value
- if genuinely unreadable → return `null`
- add a warning

Never guess merely to satisfy the schema.

---

## 19. ANTI-HALLUCINATION CONTRACT

The system prompt must explicitly instruct the model:

Extract only information supported by the document.

Never guess, invent, fabricate, or silently correct values.

If a value cannot be read reliably, return null and add a warning.

Never change a financial value merely to make arithmetic pass.

The model must not invent:

- vendor
- invoice ID
- date
- quantity
- unit price
- tax
- subtotal
- total
- line items

When uncertain:

```
value = null  
warning = appropriate explanation
```

---

## 20. COMPACT SYSTEM PROMPT

Create the model system prompt in:

```
src/document_extractor/prompts.py
```

It must contain only what the model actually needs.

Include:

- role
- extraction objective
- anti-hallucination rules
- required output behavior
- optional-field behavior
- date handling
- financial-value fidelity
- JSON-only requirement

Do not put:

- project README content
- architectural explanations
- test instructions
- developer commentary
- redundant schema descriptions

into the LLM prompt.

---

## 21. SCHEMA DESIGN

In `models.py`, create:

```
LineItem
```

Fields:

```
description: str
quantity: Decimal
unit_price: Decimal
total_price: Decimal
```



Use appropriate validation.

## ExtractionResult

Include:

```
document_type
vendor_name
date
invoice_id
currency
line_items
subtotal
discount
shipping_amount
tax_amount
total_amount
overall_confidence
warnings
```

Use optional types where the information is not guaranteed to exist.

---

## 22. DOCUMENT TYPE

Support:

```
invoice
receipt
purchase_order
unknown
```

Do not force a document into a known type when it cannot be established.

---

## 23. MONEY HANDLING

Use:

Decimal

rather than binary floating-point numbers for financial values.

Normalize common forms such as:

```
1,250.50
₹ 1,250.50
$1,250.50
EUR 100.00
```

to appropriate internal numeric values.

Use a configurable:

```
MONEY_TOLERANCE=0.01
```

for legitimate rounding differences.

Do not use unsafe locale assumptions.

---

## 24. LINE-ITEM VALIDATION

Validate:

```
quantity > 0
unit_price >= 0
total_price >= 0
```

where appropriate.

Check:

```
quantity × unit_price ≈ total_price
```

using Decimal arithmetic and configured tolerance.

Material inconsistencies must be reported.

Small currency rounding differences may pass.

---

## 25. SUBTOTAL VALIDATION

When subtotal is available:

```
sum(line_items.total_price) ≈ subtotal
```

Perform the check deterministically.

Do not ask the LLM whether it passes.

---

## 26. GRAND TOTAL VALIDATION

When sufficient components are available:

```
subtotal  
- discount  
+ shipping  
+ tax  
≈ total_amount
```

Only perform checks that are logically justified by available fields.

Do not reject every document lacking optional components.

---

## 27. DATE VALIDATION

Normalize unambiguous dates to:

```
YYYY-MM-DD
```

Support common forms such as:

```
2026-08-27  
27/08/2026  
27-08-2026
```

Aug 27, 2026  
27 Aug 2026

Do not silently resolve ambiguous dates.

For ambiguous cases:

```
date = null  
warning = date could not be determined confidently
```

Use deterministic validation after extraction.

---

## 28. PYDANTIC VALIDATION

Use Pydantic v2.

Pydantic must validate:

- schema structure
- field types
- required fields
- valid numeric representations
- valid line items
- normalized date representation

Use `@model_validator` for appropriate cross-field checks.

Do not use Pydantic as a substitute for all business logic.

Keep reusable business checks in `validators.py` where practical.

---

## 29. VALIDATION LAYERS

Use three conceptual layers:

### Layer 1 — JSON syntax

Groq JSON Object Mode + JSON parsing.

## Layer 2 — Data schema

Pydantic.

## Layer 3 — Business logic

Deterministic Python validation.

Architecture:

```
LLM
↓
JSON Syntax
↓
Pydantic Schema
↓
Business Rules
```

This separation must be documented.

---

## 30. VALIDATION STATUS

Use:

```
validated
validated_with_warnings
validation_failed
```

`validated`

Structure and relevant business checks pass.

`validated_with_warnings`

Usable extraction with non-fatal uncertainties/incomplete fields.

`validation_failed`

A meaningful deterministic consistency check failed.

Do not hide failed checks.

---

## 31. VALIDATION REPORT

Create an internal deterministic report containing checks such as:

```
schema_valid
date_valid
line_items_valid
line_item_math_valid
subtotal_valid
total_valid
document_identity_consistent
```

The UI should expose a user-friendly summary.

The LLM must not generate this report.

---

## 32. WARNING VS ERROR POLICY

Warnings:

- missing optional field
- unreadable invoice ID
- unclear vendor
- ambiguous date
- heuristic confidence concern
- some pages skipped due to configured limit
- minor non-critical inconsistency

Errors/failures:

- invalid schema
- invalid date
- impossible numeric values
- major arithmetic mismatch
- conflicting document identity
- corrupted document
- unable to obtain a usable extraction

Use clear language.

---

## 33. MULTI-PAGE MERGING

After page-level extraction:

```
Page 1
Page 2
Page 3
...
↓
Merge
↓
Deduplicate
↓
Reconcile
↓
Final validation
```

Document-level fields:

- vendor
- document ID
- date
- currency

must be reconciled.

If pages disagree:

- do not silently select one
- surface the conflict

Line items must be combined across pages.

---

## 34. DUPLICATE LINE ITEMS

Avoid duplicate line items caused by:

- repeated headers
- page carry-over
- overlapping content
- duplicate processing

Use deterministic normalized comparison.

Do not use an LLM for final deduplication.

---

## 35. PAGE DUPLICATE PROTECTION

Generate a lightweight deterministic page/content identifier or hash where appropriate.

Use it to avoid accidentally processing identical page content twice.

Do not use an LLM for this.

---

## 36. EXTRACTION MODE METADATA

Track the route:

```
image_vision  
pdf_text  
pdf_vision_fallback  
hybrid
```

For example:

```
Hybrid – 6 native-text pages + 2 vision pages
```

The UI must show this.

This is important because it visibly demonstrates the hybrid architecture.

---

## 37. PROCESSING METADATA

Track:

- number of pages
- processed pages
- skipped pages
- vision pages
- extraction mode
- processing duration
- retry count



Show appropriate information in the UI.

Do not expose sensitive internals.

---

## 38. RETRY STRATEGY

Use bounded retries.

Recommended:

```
initial request
↓
temporary API/network/rate-limit failure?
↓
one retry
↓
failure
↓
graceful error
```

Make the retry count configurable.

Do not retry infinitely.

Do not automatically retry business validation failures.

A genuine total mismatch is not necessarily an API failure.

---

## 39. API TIMEOUT

Use an explicit timeout around:

```
30 seconds
```

or another sensible configurable value.

Do not allow requests to hang indefinitely.

---

## 40. RATE-LIMIT HANDLING

Handle rate-limit errors such as HTTP 429.

When available, use server-provided retry timing/header information.

Use bounded backoff.

Do not blindly issue repeated requests.

If retry budget is exhausted:

- stop
- show a clear error
- preserve already collected page results where appropriate

---

## 41. MALFORMED JSON

Normal path:

```
response
↓
json.loads()
```

Fallback only when parsing fails:

- trim whitespace
- remove surrounding markdown code fences if present
- retry JSON parsing

Do not aggressively rewrite arbitrary content.

Do not "repair" financial numbers automatically.

If parsing still fails:

```
graceful failure
```

## 42. STREAMLIT UI

Create a clean interactive interface.

Recommended structure:

```
=====
ADVANCED DOCUMENT DATA EXTRACTOR
=====

Upload Document
[ File Uploader ]

Supported:
PDF / PNG / JPG / JPEG
Maximum: 10 MB

[ Process Document ]

-----

Uploaded Document      Extraction Result

[Document Preview]    Validation Status
                      Extraction Method
                      Pages Processed
                      Vision Pages Used
                      Confidence

                      Extracted JSON

                      Validation Checks

                      Warnings

                      [ Download JSON ]

-----
```

Use:

```
st.columns(2)
```

for the main layout.

Use:

```
st.spinner()
```

during processing.

---

## 43. UPLOAD VALIDATION

Before any processing:

1. Check file extension.
2. Check file size.
3. Validate image/PDF readability.
4. Reject invalid files.
5. Do not make an LLM call for an invalid upload.

Maximum:

```
10 MB
```

---

## 44. UI STATUS DISPLAY

Example:

✅ Validated

Document Type: Invoice  
Extraction Method: Hybrid  
Pages Processed: 4/4  
Vision Pages Used: 1  
Confidence: 0.94

Warning example:

⚠️ Validated with warnings

- Invoice ID could not be confidently read.
- Page 3 required vision fallback.

Failure example:

 Validation failed

- Line-item sum does not reconcile with subtotal.
- Grand total differs from calculated amount.

Make the reasons visible.

---

## 45. DOWNLOADABLE OUTPUT

Provide a JSON download button.

The downloaded JSON must be clean and structured.

Do not include:

- API keys
- prompts
- hidden reasoning
- stack traces
- internal logs

The UI may separately show validation metadata.

---

## 46. CLEAN JSON VS UI VALIDATION METADATA

Keep the actual extraction JSON focused on business data.

The UI can additionally display:

- validation status
- validation checks
- extraction method
- processing statistics

Do not unnecessarily pollute the business extraction schema with internal implementation details.

---

## 47. SAMPLE DOCUMENTS

The repository must contain varied layouts.

At minimum:

### Layout A — Traditional invoice

Example conceptual structure:

```
Vendor
Invoice Number
Date

Description | Qty | Unit Price | Total

Subtotal
Tax
Grand Total
```

### Layout B — Receipt-style

Example conceptual structure:

```
STORE NAME
DATE

ITEM A      2 x 100
ITEM B      1 x 250

TAX
TOTAL
```

Strongly consider also including:

### Layout C — Purchase order

with:

- supplier
- PO number
- order date
- line items

- quantities
- prices

The system must not depend on hard-coded coordinates or one fixed template.

---

## 48. SAMPLE OUTPUT REQUIREMENT

For every supplied sample document, create a corresponding extracted JSON output in:

expected\_outputs/

These must be based on **actual runs of the finished application** and manually checked.

Never invent expected outputs that do not correspond to the sample files.

---

## 49. TESTING

Use pytest.

At minimum test:

### Model tests

- valid extraction result
- invalid date
- invalid amount
- invalid quantity
- missing required field handling

### Financial tests

- valid line arithmetic
- invalid line arithmetic
- valid subtotal
- invalid subtotal
- valid total
- invalid total
- tolerance behavior

## Normalization tests

- comma-separated numbers
- currency symbols
- empty/null-like strings
- date normalization
- ambiguous dates

## Router tests

- good PDF text → text route
- poor PDF text → vision fallback

## Merge tests

- multiple pages
- line-item combination
- duplicate prevention
- document field conflict detection

## Image tests

- RGB conversion
- resizing
- invalid image handling

---

# 50. REQUIREMENTS.TXT

Pin or appropriately constrain dependency versions to improve reproducibility.

Include only dependencies actually required.

Do not add unnecessary packages.

All imports used by the application must correspond to installed dependencies.

---

# 51. ENVIRONMENT FILE

Create:

```
.env.example
```



with:

```
GROQ_API_KEY=your_groq_api_key_here
GROQ_MODEL=qwen/qwen3.6-27b

MAX_FILE_SIZE_MB=10
MAX_PDF_PAGES=10
MAX_VISION_PAGES=5
MAX_IMAGE_DIMENSION=1024

API_TIMEOUT_SECONDS=30
MAX_RETRIES=1
MAX_COMPLETION_TOKENS=<sensible_value>

MONEY_TOLERANCE=0.01
TEXT_QUALITY_THRESHOLD=<sensible_value>
```

Choose technically sensible defaults.

Do not commit the real `.env`.

---

## 52. GITIGNORE

At minimum:

```
.env
.venv/
__pycache__/
*.pyc
.pytest_cache/
.streamlit/secrets.toml
```

---

## 53. SECURITY

Treat uploads as untrusted.

Implement:

- extension validation
- size validation

- safe file opening
- safe PDF handling
- safe image handling
- environment-based secrets
- no credential logging

Never expose secrets in source code or README.

---

## 54. README — PRIMARY REQUIREMENT

The README must be comprehensive enough that a stranger can clone the repository onto another machine and run the project without studying the source code first.

Include these sections:

```
# Document Data Extractor

## What It Does

## Features

## Architecture

## How the Hybrid Pipeline Works

## Project Structure

## Where To Make Changes

## Tech Stack

## Requirements

## Setup & Installation

## Environment Variables

## How To Run

## How To Test

## Sample Documents

## Expected Output
```

```
## Validation Logic

## Multi-Page Processing

## Free-Tier Optimization

## Error Handling

## Known Failure Cases

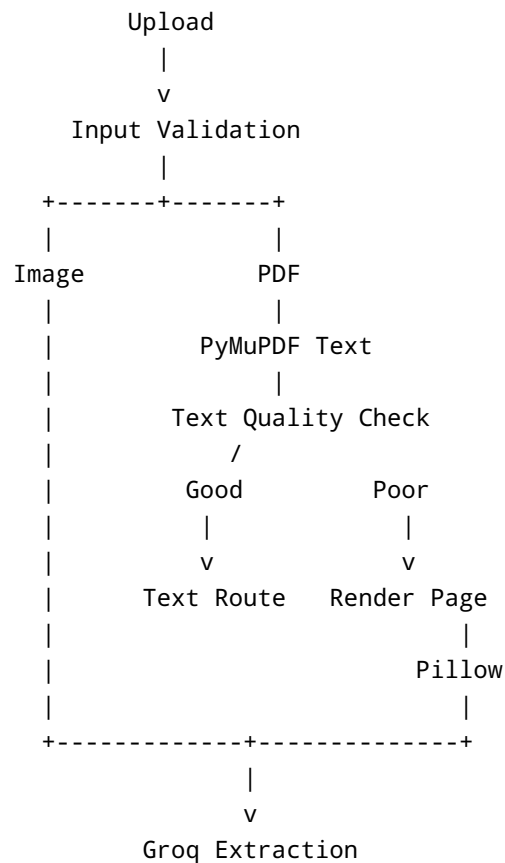
## Tradeoffs & Future Improvements

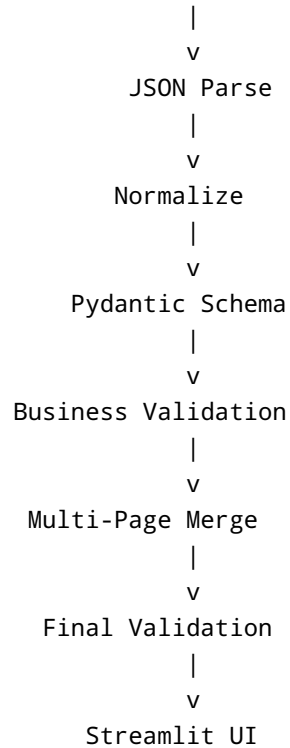
## Limitations

## Deployment
```

## 55. README ARCHITECTURE

The README MUST contain an architecture diagram such as:





The architecture section must explain what each stage does.

## 56. README — WHERE TO MAKE CHANGES

This section is mandatory.

Explain exactly where future developers should edit the project.

Example:

Need to change the LLM/model?

- `src/document_extractor/extractor.py`
- `src/document_extractor/prompts.py`
- `.env`

Need to change the JSON schema?

- `src/document_extractor/models.py`

Need to change financial sanity checks?

- `src/document_extractor/validators.py`

Need to change PDF processing?

→ src/document\_extractor/pdf\_processor.py

Need to change image preprocessing?

→ src/document\_extractor/image\_processor.py

Need to change text-vs-vision routing?

→ src/document\_extractor/text\_router.py

Need to change number/date normalization?

→ src/document\_extractor/normalizer.py

Need to change multi-page merging?

→ src/document\_extractor/merger.py

Need to change the Streamlit UI?

→ src/document\_extractor/app.py

Need to change configuration?

→ .env

→ src/document\_extractor/config.py

Need to change model instructions?

→ src/document\_extractor/prompts.py

Need to add/change tests?

→ tests/

Make this match the actual implementation exactly.

---

## 57. README — SETUP ON ANOTHER SYSTEM

Provide commands for Windows and macOS/Linux.

Example Windows:

```
git clone <YOUR_REPOSITORY_URL>
cd document-data-extractor
```

```
python -m venv .venv
.venv\Scripts\activate
```

```
pip install -r requirements.txt
```

```
copy .env.example .env
```

Then configure:

```
GROQ_API_KEY=your_key_here
```

Run:

```
streamlit run src/document_extractor/app.py
```

Test:

```
pytest
```

macOS/Linux:

```
git clone <YOUR_REPOSITORY_URL>
cd document-data-extractor

python3 -m venv .venv
source .venv/bin/activate

pip install -r requirements.txt

cp .env.example .env
```

Run:

```
streamlit run src/document_extractor/app.py
```

Test:

```
pytest
```

## 58. README — VALIDATION LOGIC

Clearly explain:

### **Date validation**

How dates are normalized and why ambiguous dates are not guessed.

### **Line-item validation**

How:

$$\text{quantity} \times \text{unit\_price} \approx \text{total\_price}$$

is checked.

### **Subtotal validation**

How line-item sums are reconciled.

### **Grand-total validation**

How subtotal, discount, shipping, and tax are reconciled.

### **Decimal/tolerance logic**

Explain why Decimal and tolerance are used.

### **Multi-page reconciliation**

Explain how document-level fields and line items are combined.

---

## 59. README — KNOWN FAILURE CASES

Include honest limitations such as:

- very low-resolution scans
- severe blur
- handwritten documents
- ambiguous dates
- unusual table layouts
- visually overlapping text
- OCR/vision digit confusion

- decimal-point confusion
- inconsistent source documents
- unsupported document types
- PDFs exceeding configured processing limits
- rate-limit exhaustion
- API service interruption

Do not claim perfect accuracy.

---

## 60. README — FREE-TIER OPTIMIZATION

Explicitly explain:

- native PDF text is preferred when usable
- vision is only a fallback
- one page per vision request
- page budgets exist
- retries are bounded
- reasoning is disabled
- output is compact
- validation is performed locally
- no LLM is used for arithmetic or date validation

Do not claim unsupported token savings.

---

## 61. README — TRADEOFFS

Explain:

### **Vision vs native text**

Native text is cheaper but may lose layout relationships.

Vision better handles messy/scanned layouts but consumes more API resources.

### **Free-tier limits**

Page and vision budgets intentionally restrict expensive processing.

### **Model choice**

Qwen 3.6 27B was selected for multimodal extraction and current Groq availability within this project's constraints.



## Validation

Pydantic and deterministic business checks catch structural/math issues but cannot recover unreadable source information.

## Hybrid routing

Routing improves efficiency but introduces a text-quality classification threshold that may occasionally choose the wrong route.

## Preview model

Keep the model configurable because model availability/capabilities may change.

---

# 62. README — TRADEOFF BETWEEN SPEED, COST, AND ACCURACY

Explicitly state that the architecture favors:

```
reasonable accuracy
+
controlled free-tier cost
+
simple maintainability
```

rather than unlimited inference.

Explain why every page is not automatically sent to vision.

---

# 63. DEPLOYMENT

Document how a stranger can deploy the Streamlit application.

Explain that:

- `.env` is for local development
- hosted deployment should use platform secrets/environment variables
- `GROQ_API_KEY` must never be committed

Do not introduce unnecessary infrastructure.

---

## 64. LOGGING

Use lightweight Python logging.

Log useful events:

- processing started
- selected extraction route
- page number
- retry count
- validation result
- total processing duration

Do not log:

- API keys
  - secrets
  - unnecessary raw financial document contents
  - hidden reasoning
- 

## 65. ERROR HANDLING

Gracefully handle:

- missing API key
- unsupported file
- oversized file
- corrupt PDF
- corrupt image
- API timeout
- rate limit
- API errors
- malformed JSON
- Pydantic validation errors
- financial validation errors
- partial PDF processing

Normal users should not receive raw Python tracebacks.

---

## 66. FUTURE EXTENSIBILITY

Design the architecture so future additions can be made with minimal changes:

- another LLM provider
- another Groq model
- OCR fallback
- batch processing
- API backend
- database persistence
- field-level confidence
- human review
- additional document types
- currency-specific rules

Do not implement these now unless required.

The current architecture should permit them cleanly.

---

## 67. PROVIDER SEPARATION

Keep Groq-specific code concentrated in:

```
src/document_extractor/extractor.py
```

and model instructions in:

```
src/document_extractor/prompts.py
```

The rest of the pipeline should not depend heavily on Groq-specific internals.

This is important for future provider/model replacement.

---

## 68. NO UNNECESSARY AGENT FRAMEWORKS

Do not use:

- LangChain
- LangGraph
- CrewAI

- AutoGen
- vector databases
- RAG
- SQL database
- external OCR service
- multi-agent systems

The task is focused extraction.

The agent loop is:

```
Input
↓
Understand
↓
Extract
↓
Normalize
↓
Validate
↓
Output
```

That is sufficient.

---

## 69. OUTPUT CONTRACT

The core business JSON should look conceptually like:

```
{
  "document_type": "invoice",
  "vendor_name": "Example Vendor",
  "date": "2026-08-27",
  "invoice_id": "INV-1001",
  "currency": "USD",
  "line_items": [
    {
      "description": "Product A",
      "quantity": "2",
      "unit_price": "25.00",
      "total_price": "50.00"
    }
  ],
}
```

```
"subtotal": "50.00",
"discount": "0.00",
"shipping_amount": "0.00",
"tax_amount": "5.00",
"total_amount": "55.00",
"overall_confidence": 0.95,
"warnings": []
}
```

Use consistent JSON serialization.

Decimal values should be serialized safely, preferably as strings.

Do not expose internal implementation fields inside the business extraction object unless explicitly required.

---

## 70. FINAL QUALITY PRINCIPLES

The final implementation must follow:

**Evidence over guessing**

**Deterministic validation over LLM self-validation**

**Local processing before API processing**

**Minimal API requests**

**Minimal output tokens**

**Modular code over monolithic scripts**

**Clear documentation over assumptions**

**Honest limitations over exaggerated claims**

---

## 71. FINAL ACCEPTANCE CRITERIA

Before presenting the completed project, verify all of the following:

```
[ ] Project is package-structured under src/
[ ] README exists
```

```
[ ] docs/architecture.md exists
[ ] README contains architecture
[ ] README contains "Where To Make Changes"
[ ] README contains exact setup commands
[ ] README contains exact run commands
[ ] README contains exact test commands
[ ] README contains validation logic
[ ] README contains known failure cases
[ ] README contains tradeoffs
[ ] README contains free-tier optimization explanation

[ ] Groq is used
[ ] qwen/qwen3.6-27b is used
[ ] model is configurable
[ ] API key is environment-based
[ ] .env is ignored
[ ] .env.example exists

[ ] Streamlit UI exists
[ ] PDF input exists
[ ] PNG input exists
[ ] JPG/JPEG input exists
[ ] 10 MB upload limit exists

[ ] PyMuPDF is used
[ ] Native PDF text extraction happens first
[ ] Text-quality router exists
[ ] Vision fallback exists
[ ] Poor PDF text can trigger vision fallback
[ ] Image preprocessing exists
[ ] 1024px max edge is configurable
[ ] One vision page per LLM request
[ ] MAX_PDF_PAGES exists
[ ] MAX_VISION_PAGES exists

[ ] reasoning_effort="none"
[ ] max_completion_tokens is bounded
[ ] JSON Object Mode is used
[ ] full Pydantic schema is NOT injected into every request
[ ] optional unavailable fields can be omitted
[ ] anti-hallucination rules exist
[ ] malformed JSON fallback exists
[ ] timeout exists
[ ] bounded retry exists
[ ] rate-limit handling exists

[ ] Pydantic v2 exists
[ ] Decimal is used for financial values
```

```
[ ] date validation exists
[ ] line-item validation exists
[ ] subtotal validation exists
[ ] total reconciliation exists
[ ] warnings exist
[ ] validation status exists
[ ] deterministic validation report exists

[ ] multi-page merge exists
[ ] document-level reconciliation exists
[ ] line-item aggregation exists
[ ] duplicate prevention exists
[ ] page duplicate protection exists

[ ] extraction method is visible in UI
[ ] pages processed visible
[ ] vision pages visible
[ ] confidence visible
[ ] validation checks visible
[ ] warnings visible
[ ] JSON download exists

[ ] at least two different layouts are provided
[ ] sample documents exist
[ ] expected outputs exist
[ ] expected outputs correspond to actual sample files

[ ] pytest tests exist
[ ] routing tests exist
[ ] normalization tests exist
[ ] validation tests exist
[ ] merge tests exist

[ ] no unnecessary frameworks
[ ] no hard-coded credentials
[ ] no unsupported accuracy claims
[ ] all imports exist in requirements
[ ] all documentation matches implementation
```

---

## 72. MOST IMPORTANT IMPLEMENTATION RULE

Do not optimize for theoretical complexity.

Optimize for:

```
A reviewer can clone it
↓
install it
↓
configure one API key
↓
run one command
↓
upload a document
↓
see extraction
↓
see validation
↓
download JSON
↓
understand how the code works
↓
modify one component without breaking everything
```

The result should look like a **small, thoughtfully engineered document-processing system**, not merely an LLM wrapper.

The final submission must explicitly satisfy the challenge's required deliverables:

## Output Requirements

- **Validated, well-structured JSON**
- **Sanity checks including line-item arithmetic and valid dates**
- **At least two different document layouts**

## Agent-Specific Deliverables

- **Sample documents with varied layouts**
- **Extracted JSON for each sample document**
- **Documentation explaining validation logic and known failure cases**

Do not omit any of these requirements when generating the final codebase.